Python important Q&A

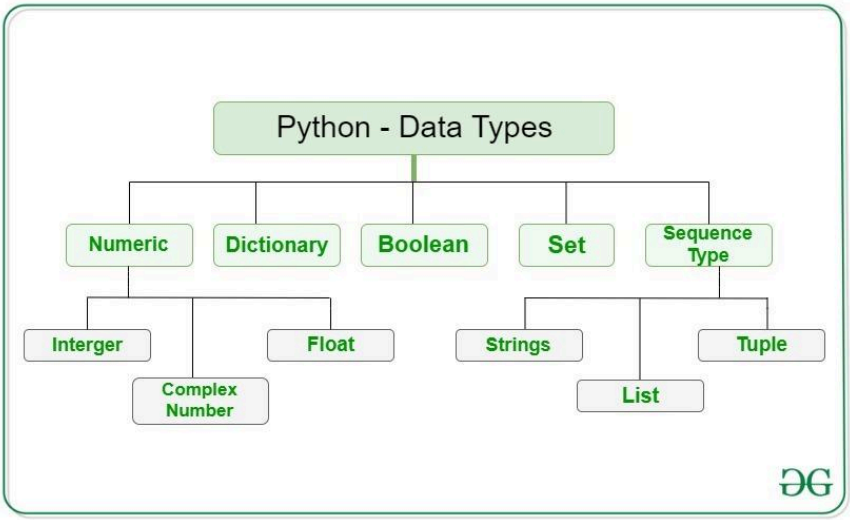
1.What is Python?

Python is a high-level, interpreted, and general-purpose programming language. It is known for its easy-to-read syntax and versatility, making it suitable for various applications, including web development, data analysis, and artificial intelligence.

2.What are the key features of Python?

- Easy syntax and readability
- Interpreted and dynamically typed
- Supports object-oriented, functional, and procedural programming
- Large standard library
- Cross-platform compatibility

3.What are Python's data types?



There are several built-in data types in Python, including:

- Numeric types: int, float, complex
- Sequence types: list, tuple, range

- Mapping type: dict
- Set types: set, frozenset
- Text type: str
- Boolean type: bool
- Binary types: bytes, bytearray, memoryview

4.What are lists and tuples in Python?

List in Python: Lists are mutable (modifiable) ordered collections in Python that can hold a variety of object types, including integers, strings, and other lists. They allow for dynamic resizing.

Tuples in Python: Tuples are immutable (non-modifiable) ordered collections in Python. Once created, the elements in a tuple cannot be changed, making them suitable for storing fixed data.

5.What is a dictionary in Python?

A dictionary in Python is a collection of key-value pairs where each key is unique. It is unordered, mutable, and indexed. It allows fast lookups and is defined using curly braces { }.

6.What are Lambda functions?

A lambda function in Python is a small anonymous function defined using the lambda keyword. It can take any number of arguments but only has one expression.

```
# A lambda function to find the square of a number

square = lambda x: x ** 2

print("Square of 5:", square(5))
```

Output Square of 5: 25

7.What is the difference between is and == in Python?

The difference between is and == in Python

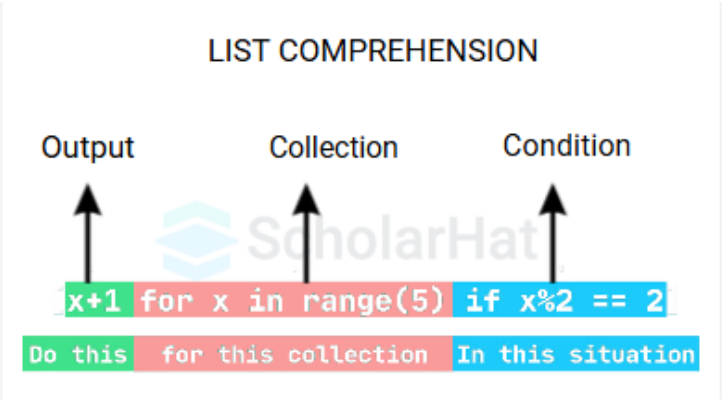
- == checks if the values of two objects are equal.
- is checks if two objects refer to the same memory location (i.e., they are the same object).

8.What is the difference between append() and extend() in Python?

The difference between append() and extend() in Python:

- append() adds a single element to the end of a list.
- extend() adds all elements from an iterable to the end of the list.

9.What are list comprehensions in Python?



List comprehensions in Python provide a concise way to create lists. They consist of an expression followed by a for clause, optionally including **if** conditions.

```
# List of numbers

numbers = [1, 2, 3, 4, 5]

# List comprehension to square each number

squared_numbers = [x ** 2 for x in numbers]

print("Squared Numbers:", squared_numbers)
```

Output

Squared Numbers: [1, 4, 9, 16, 25]

10.Is Python a compiled language or an interpreted language?

Please remember one thing, whether a language is compiled or interpreted or both is not defined in the language standard. In other words, it is not a properly of a programming language. Different Python distributions (or implementations) choose to do different things (compile or interpret or both). However the most common implementations like CPython do both compile and interpret, but in different stages of its execution process.

- **Compilation:** When you write Python code and run it, the source code (.py files) is first compiled into an intermediate form called bytecode (.pyc files). This bytecode is a lower-level representation of your code, but it is still not directly machine code. It's something that the Python Virtual Machine (PVM) can understand and execute.
- **Interpretation:** After Python code is compiled into bytecode, it is executed by the Python Virtual Machine (PVM), which is an interpreter. The PVM reads the bytecode and executes it line-by-line at runtime, which is why Python is considered an interpreted language in practice.

Some implementations, like PyPy, use Just-In-Time (JIT) compilation, where Python code is compiled into machine code at runtime for faster execution, blurring the lines between interpretation and compilation.

11.How can you concatenate two lists in Python?

We can concatenate two lists in Python using the +operator or the extend() method.

1. Using the + operator:

```
a = [1, 2, 3]

b = [4, 5, 6]

res = a + b

print(res)
```

2. Using the extend() method:

```
a = [1, 2, 3]

b = [4, 5, 6]

a.extend(b)

print(a)
```

12.Difference between for loop and while loop in Python

- For loop: Used when we know how many times to repeat, often with lists, tuples, sets, or dictionaries.
- While loop: Used when we only have an end condition and don't know exactly how many times it will repeat.

13.What is the difference between / and // in Python?

/ represents precise division (result is a floating point number) whereas // represents floor division (result is an integer). For Example:

```
print(5//2)

print(5/2)
```

Output

```
2
2.5
```

14.What is a dynamically typed language?

- In a dynamically typed language, the data type of a variable is determined at runtime, not at compile time.
- No need to declare data types manually; Python automatically detects it based on the assigned value.
- Examples of dynamically typed languages: Python, JavaScript.
- Examples of statically typed languages: C, C++, Java.
- Dynamically typed languages are easier and faster to code.
- Statically typed languages are usually faster to execute due to type checking at compile time.

15.What is the difference between a Set and Dictionary?

- A Python Set is an unordered collection data type that is iterable, mutable and has no duplicate elements. Python's set class represents the mathematical notion of a set.
- Syntax: Defined using curly braces {} or the set() function.

```
my_set = {1, 2, 3}
```

- Dictionary in Python is an ordered (since Py 3.7) [unordered (Py 3.6 & prior)] collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds key:value pair. Key-value is provided in the dictionary to make it more optimized.
- Syntax: Defined using curly braces {} with key-value pairs.

```
my_dict = {"a": 1, "b": 2, "c": 3}
```

16.What are Built-in data types in Python?

The following are the standard or built-in data types in Python:

- Numeric: The numeric data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, a Boolean, or even a complex number.
- Sequence Type: The sequence Data Type in Python is the ordered collection of similar or different data types. There are several sequence types in Python:
    - Python String
    - Python List
    - Python Tuple
    - Python range
- Mapping Types: In Python, hashable data can be mapped to random objects using a mapping object. There is currently only one common mapping type, the dictionary and mapping objects are mutable.
    - Python Dictionary
- Set Types: In Python, a Set is an unordered collection of data types that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

17. What is the difference between a Mutable datatype and an Immutable data type?

- Mutable data types can be edited i.e., they can change at runtime. Eg – List, Dictionary, etc.
- Immutable data types can not be edited i.e., they can not change at runtime. Eg – String, Tuple, etc.

18.What is a Variable Scope in Python?

The location where we can find a variable and also access it if required is called the scope of a variable.

- Python Local variable: Local variables are those that are initialized within a function and are unique to that function. A local variable cannot be accessed outside of the function.
- Python Global variables: Global variables are the ones that are defined and declared outside any function and are not specified to any function.
- Module-level scope: It refers to the global objects of the current module accessible in the program.
- Outermost scope: It refers to any built-in names that the program can call. The name referenced is located last among the objects in this scope.

19. How is a dictionary different from a list?

A list is an ordered collection of items accessed by their index, while a dictionary is an unordered collection of key-value pairs accessed using unique keys. Lists are ideal for sequential data, whereas dictionaries are better for associative data. For example, a list can store [10, 20, 30], whereas a dictionary can store {"a": 10, "b": 20, "c": 30}.

20. What is docstring in Python?

Python documentation strings (or docstrings) provide a convenient way of associating documentation with Python modules, functions, classes and methods.

- Declaring Docstrings: The docstrings are declared using ”’triple single quotes”’ or “”””triple double quotes””” just below the class, method, or function declaration. All functions should have a docstring.
- Accessing Docstrings: The docstrings can be accessed using the __doc__ method of the object or using the help function.

21.What is the difference between Python Arrays and Lists?

- Arrays (when talking about the array module in Python) are specifically used to store a collection of numeric elements that are all of the same type. This makes them more efficient for storing large amounts of data and performing numerical computations where the type consistency is maintained.
- Syntax: Need to import the array module to use arrays.

Example:

```
from array import array
arr = array('i', [1, 2, 3, 4])  # Array of integers
print (arr)
```

```
array('i', [1, 2, 3, 4])
```

Output

- Lists are more flexible than arrays in that they can hold elements of different types (integers, strings, objects, etc.). They come built-in with Python and do not require importing any additional modules.
- Lists support a variety of operations that can modify the list.

Example:

```
a = [1, 'hello', 3.14, [1, 2, 3]]
print (a)
```

```
[1, 'hello', 3.14, [1, 2, 3]]
```

22.Differentiate between List and Tuple?

Let's analyze the differences between List and Tuple:

List

- Lists are Mutable datatype.

- Lists consume more memory
- The list is better for performing operations, such as insertion and deletion.
- The implication of iterations is Time-consuming

Tuple

- Tuples are Immutable datatype.
- Tuple consumes less memory as compared to the list
- A Tuple data type is appropriate for accessing the elements
- The implication of iterations is comparatively Faster

23.What is slicing in Python?

Python Slicing is a string operation for extracting a part of the string, or some part of a list. With this operator, one can specify where to start the slicing, where to end and specify the step. List slicing returns a new list from the existing list.

Syntax: substring = s[start : end : step]

Example:

```
b = "Hello, World!"
print(b[2:5])
o.p: llo


b = "Hello, World!"
print(b[:5])
O.p: Hello


b = "Hello, World!"
print(b[2:])
O.p:llo, World!
```

24.What is break, continue and pass in Python?

The break statement terminates the loop immediately and the control flows to the statement after the body of the loop.

The continue statement terminates the current iteration of the statement, skips the rest of the code in the current iteration and the control flows to the next iteration of the loop.
The pass keyword in Python is generally used to fill up empty blocks and is similar to an empty statement represented by a semi-colon in languages such as Java, C++, Javascript, etc.

```
pat = [1, 3, 2, 1, 2, 3, 1, 0, 1, 3]
for p in pat:
    pass
    if (p == 0):
        current = p
        break
    elif (p % 2 == 0):
        continue
    print(p)    # output => 1 3 1 3 1
print(current)    # output => 0
```

25. What are negative indexes and why are they used?

- Negative indexes are the indexes from the end of the list or tuple or string.
- Arr[-1] means the last element of array Arr[]

```
arr = [1, 2, 3, 4, 5, 6]
#get the last element
print(arr[-1]) #output 6
#get the second last element
print(arr[-2]) #output 5
```

26.Explain split() and join() functions in Python?

- You can use split() function to split a string based on a delimiter to a list of strings.
- You can use join() function to join a list of strings based on a delimiter to give a single string.

```
string = "This is a string."
string_list = string.split(' ') #delimiter is ‘space’ character or ‘ ‘
print(string_list) #output: ['This', 'is', 'a', 'string.']
print(' '.join(string_list)) #output: This is a string.
```

27. How do you define a function in Python?

Answer: You define a function in Python using the `def` keyword followed by the function name and a set of parentheses containing parameters. The function body is indented below.

```
def my_function(parameter1, parameter2):
# Function body
# ...
```

28.What is a docstring, and why is it used in Python functions?

Answer: A docstring is a string literal used to provide documentation for functions, classes, or modules. It helps users understand the purpose and usage of the function.

```
def my_function(parameter):
"""
This is a docstring.
It provides documentation for the function.
"""
# Function body
# ...
```

29. What is the purpose of the `None` keyword in Python functions?

Answer: `None` is used to represent the absence of a value in functions. Functions without a `return` statement implicitly return `None`.

30.What is the purpose of the `*args` and `**kwargs` parameters in Python functions?

Answer: `*args` is used to pass a variable number of non-keyword arguments to a function, while `**kwargs` is used to pass a variable number of keyword arguments.

```
def my_function(arg1, *args, kwarg1="default", **kwargs):
# arg1 is a regular argument
# *args is a tuple of additional non-keyword arguments
# kwarg1 is a keyword argument with a default value
# **kwargs is a dictionary of additional keyword arguments
```

31.**How can you access global variables within a function in Python?** – You can access global variables within a function by using the `global` keyword or by referencing them directly without reassignment.

32. Explain the purpose of the `lambda` keyword in Python. – The `lambda` keyword is used to create anonymous (nameless) functions, often for simple operations. It returns a function object.

33.**Explain the purpose of the `map()` function in Python.** – The `map()` function applies a specified function to each item in an iterable and returns an iterator of the results.

34. How can you define a generator function in Python? – A generator function is defined like a regular function but contains one or more `yield` statements. It generates values lazily and is often used with large datasets.

35. What is a decorator in Python, and how is it used with functions? – A decorator is a higher-order function that can be used to modify or enhance the behavior of another function. It is applied using the `@decorator_name` syntax.

36.What is the purpose of the `global` keyword in Python functions, and when should you use it? – The `global` keyword is used to indicate that a variable declared within a function should refer to a global variable instead of creating a new local variable. It should be used sparingly to modify global variables within a function.

37.What is NumPy? Why should we use it?

NumPy (also called Numerical Python) is a highly flexible, optimized, open-source package meant for array processing. It provides tools for delivering high-end performance while dealing with N-dimensional powerful array objects. It is also beneficial for performing scientific computations, mathematical, and logical operations, sorting operations, I/O functions, basic statistical and linear algebra-based operations along with random simulation and broadcasting functionalities. Due to the vast range of capabilities, NumPy has become very popular and is the most preferred package.

38.How do you concatenate 2 NumPy arrays?

Concatenating 2 arrays by adding elements to the end can be achieved by making use of the concatenate() method of the NumPy package. Syntax:

```
np.concatenate((a1, a2, ...), axis=0, out=None)
```

where,

- a1,a2: arrays of the same shape
- axis: Represents the axis along which the arrays are joined. The default value is 0.
- out: If mentioned, it specifies the destination for placing the result.

39.How is arr[:,0] different from arr[:,[0]]

arr[:,0] - Returns 0th index elements of all rows. In other words, return the first column elements.

```
import numpy as np
arr = np.array([[1,2,3,4],[5,6,7,8]])
new_arr =arr[:,0]
print(new_arr)
```

Output:

```
[1 5]
```

arr[:,[0]] - This returns the elements of the first column by adding extra dimension to it.

```
import numpy as np
arr = np.array([[1,2,3,4],[5,6,7,8]])
new_arr =arr[:,[0]]
print(new_arr)
```

Output:

```
[[1]
[5]]
```

40. What are ndarrays in NumPy?

ndarray object is the core of the NumPy package. It consists of n-dimensional arrays storing elements of the same data types and also has many operations that are done in compiled code for optimised performance. These arrays have fixed sizes defined at the time of creation. Following are some of the properties of ndarrays:

- When the size of ndarrays is changed, it results in a new array and the original array is deleted.
- The ndarrays are bound to store homogeneous data.
- They provide functions to perform advanced mathematical operations in an efficient manner.

41.How are NumPy arrays better than Python's lists?

- Python lists support storing heterogeneous data types whereas NumPy arrays can store datatypes of one nature itself. NumPy provides extra functional capabilities that make operating on its arrays easier which makes NumPy array advantageous in comparison to Python lists as those functions cannot be operated on heterogeneous data.
- NumPy arrays are treated as objects which results in minimal memory usage. Since Python keeps track of objects by creating or deleting them based on the requirements, NumPy objects are also treated the same way. This results in lesser memory wastage.
- NumPy arrays support multi-dimensional arrays.
- NumPy provides various powerful and efficient functions for complex computations on the arrays.
- NumPy also provides various range of functions for BitWise Operations, String Operations, Linear Algebraic operations, Arithmetic operations etc. These are not provided on Python's default lists.

42. How can you reverse a NumPy array?

There are two ways of reversing a NumPy array.

- Method 1: Using the slicing method: We can make use of [::-1] for reversing the array. The following example demonstrates this:

```
import numpy as np

# create numpy array
arr = np.array([1, 2, 4, 6])

# To reverse array
reverse_arr = arr[::-1]
print(reverse_arr)
```

Output: [6 4 2 1]

- Method 2: flipud function: This function is provided by NumPy to reverse the NumPy array. Let us see the below example about its usage.

```
import numpy as np

# create numpy array
arr = np.array([1, 2, 4, 5, 6])

#flipud method for reversing
reverse_arr = np.flipud(arr)
print(reverse_arr)
```

Output: [6 5 4 2 1]

43.What are exceptions in Python?

Answer: Exceptions are events that occur during the execution of a program, disrupting the normal flow of instructions. They are used to handle errors and unexpected situations.

44. How do you handle exceptions using try, except, and finally blocks?

Answer: You use a `try` block to enclose code that may raise an exception. If an exception occurs, the `except` block is executed to handle it. The `finally` block, if present, is executed afterward regardless of whether an exception occurred.

45.What is the difference between the `except` and `else` blocks in exception handling?

Answer: The `except` block is executed when an exception occurs, and you specify the exception type to catch. The `else` block is executed if no exceptions are raised in the `try` block.

46.Explain the difference between a built-in exception and a custom exception.

Answer: Built-in exceptions are provided by Python and cover a wide range of common error conditions. Custom exceptions are user-defined and are tailored to specific application requirements.

47.What is the difference between the `except` and `finally` blocks in exception handling?

Answer: The `except` block is used to handle exceptions. The `finally` block is used for cleanup code that always runs, regardless of whether an exception occurred or not.

48.What are the main features of OOPs?

OOPs or Object Oriented Programming mainly comprises of the below four features, and make sure you don't miss any of these:

- Inheritance
- Encapsulation
- Polymorphism
- Data Abstraction

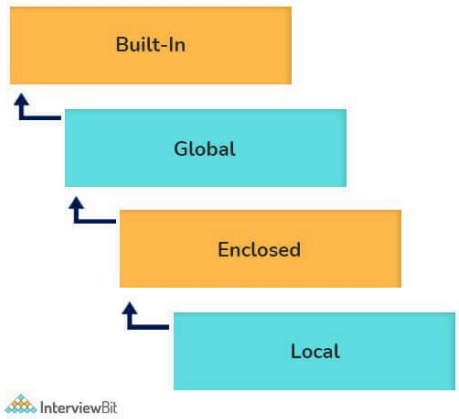
49. What are Python namespaces? Why are they used?

A namespace in Python ensures that object names in a program are unique and can be used without any conflict. Python implements these namespaces as dictionaries with 'name as key' mapped to a corresponding 'object as value'. This allows for multiple namespaces to use the same name and map it to a separate object. A few examples of namespaces are as follows:

- **Local Namespace** includes local names inside a function. the namespace is temporarily created for a function call and gets cleared when the function returns.

- **Global Namespace** includes names from various imported packages/ modules that are being used in the current project. This namespace is created when the package is imported in the script and lasts until the execution of the script.
- **Built-in Namespace** includes built-in functions of core Python and built-in names for various types of exceptions.

The lifecycle of a namespace depends upon the scope of objects they are mapped to. If the scope of an object ends, the lifecycle of that namespace comes to an end. Hence, it isn't possible to access inner namespace objects from an outer namespace.



50. What is the difference between xrange and range in Python?

xrange() and range() are quite similar in terms of functionality. They both generate a sequence of integers, with the only difference that range() returns a Python list, whereas, xrange() returns an xrange object.

So how does that make a difference? It sure does, because unlike range(), xrange() doesn't generate a static list, it creates the value on the go. This technique is commonly used with an object-type generator and has been termed as "yielding".

Yielding is crucial in applications where memory is a constraint. Creating a static list as in range() can lead to a Memory Error in such conditions, while, xrange() can handle it optimally by using just enough memory for the generator (significantly less in comparison).

```
for i in xrange(10):    # numbers from o to 9
  print i       # output => 0 1 2 3 4 5 6 7 8 9
```

```
for i in xrange(1,10):    # numbers from 1 to 9
  print i       # output => 1 2 3 4 5 6 7 8 9
for i in xrange(1, 10, 2):    # skip by two for next
  print i       # output => 1 3 5 7 9
```

Note: xrange has been deprecated as of Python 3.x. Now range does exactly the same as what xrange used to do in Python 2.x, since it was way better to use xrange() than the original range() function in Python 2.x

51. How do you copy an object in Python?

In Python, the assignment statement (= operator) does not copy objects. Instead, it creates a binding between the existing object and the target variable name. To create copies of an object in Python, we need to use the copy module. Moreover, there are two ways of creating copies for the given object using the copy module -

Shallow Copy is a bit-wise copy of an object. The copied object created has an exact copy of the values in the original object. If either of the values is a reference to other objects, just the reference addresses for the same are copied.
Deep Copy copies all values recursively from source to target object, i.e. it even duplicates the objects referenced by the source object.

```
from copy import copy, deepcopy
list_1 = [1, 2, [3, 5], 4]
## shallow copy
list_2 = copy(list_1)
list_2[3] = 7
list_2[2].append(6)
list_2    # output => [1, 2, [3, 5, 6], 7]
list_1    # output => [1, 2, [3, 5, 6], 4]
## deep copy
list_3 = deepcopy(list_1)
list_3[3] = 8
list_3[2].append(7)
list_3    # output => [1, 2, [3, 5, 6, 7], 8]
list_1    # output => [1, 2, [3, 5, 6], 4]
```